

Lab 3: Functions and Recursion

1 Functions in Python

Functions are a way to encapsulate code into reusable blocks. In Python, functions are defined using the `def` keyword.

1.1 Example 1: Function to Find the Maximum of Two Numbers: –

```
1 def find_max(a, b):
2     if a > b:
3         return a
4     else:
5         return b
6
7 # Calling the function and printing the result
8 result = find_max(10, 20)
9 print("Maximum:", result)
```

Listing 1: Find the Maximum of Two Numbers

Dry Run Explanation:

- **Function Definition:**

- `def find_max(a, b):` defines a function named `find_max` that takes two parameters `a` and `b`.

- **Function Call:**

- `result = find_max(10, 20)` calls the function with `a = 10` and `b = 20`.

- **Execution:**

- The function checks if `a > b`.
- Since `10 > 20` is `False`, it returns `b`, which is `20`.

- **Output:**

- The returned value `20` is stored in `result`.
- `print("Maximum:", result)` prints `Maximum: 20`.

1.2 Example 2: Function to Calculate the Sum of a List: –

```
1 def sum_list(numbers):
2     total = 0
3     for num in numbers:
4         total += num
5     return total
6
7 # Calling the function and printing the result
8 result = sum_list([1, 2, 3, 4, 5])
9 print("Sum:", result)
```

Listing 2: Calculate the Sum of a List

Dry Run Explanation:

PREPARED BY: Sahil Wagh (M.Tech)

- **Function Definition:**

- `def sum_list(numbers):` defines a function named `sum_list` that takes a list `numbers`.

- **Function Call:**

- `result = sum_list([1, 2, 3, 4, 5])` calls the function with `numbers = [1, 2, 3, 4, 5]`.

- **Execution:**

- `total` is initialized to 0.
- The loop iterates over each element in the list:
 - * `total += 1` → `total = 1`
 - * `total += 2` → `total = 3`
 - * `total += 3` → `total = 6`
 - * `total += 4` → `total = 10`
 - * `total += 5` → `total = 15`
- The function returns `total`, which is 15.

- **Output:**

- The returned value 15 is stored in `result`.
- `print("Sum:", result)` prints `Sum: 15`.

1.3 Example 3: Function to Check if a Number is Even or Odd: –

```
1 def is_even(number):  
2     if number % 2 == 0:  
3         return True  
4     else:  
5         return False  
6  
7 # Calling the function and printing the result  
8 result = is_even(7)  
9 print("Is 7 even?", result)
```

Listing 3: Check if a Number is Even or Odd

Dry Run Explanation:

- **Function Definition:**

- `def is_even(number):` defines a function named `is_even` that takes a parameter `number`.

- **Function Call:**

- `result = is_even(7)` calls the function with `number = 7`.

- **Execution:**

- The function checks if `number % 2 == 0`.
- Since `7 % 2 == 0` is `False`, it returns `False`.

- **Output:**

- The returned value `False` is stored in `result`.
- `print("Is 7 even?", result)` prints `Is 7 even?: False`.

2 Linear Search in Python

Linear search is a simple searching algorithm that checks each element in the list until the desired element is found or the list ends.

2.1 Example: Linear Search Implementation: –

```
1 def linear_search(arr, target):
2     for i in range(len(arr)):
3         if arr[i] == target:
4             return i # Return the index of the target element
5     return -1 # Return -1 if the target is not found
6
7 # Example usage
8 arr = [10, 20, 30, 40, 50]
9 target = 30
10 result = linear_search(arr, target)
11 print("Element found at index:", result)
```

Listing 4: Linear Search in Python

2.2 Dry Run Explanation:: Let's walk through the code step by step:

1. Initial Call: `linear_search([10, 20, 30, 40, 50], 30)`

- The function is called with `arr = [10, 20, 30, 40, 50]` and `target = 30`.
- A loop starts, iterating over each element in the array with the index `i`.

2. First Iteration:

- **Index:** `i = 0`
- **Element:** `arr[0] = 10`
- The condition `arr[i] == target` is checked. Here, `10 == 30` is `False`, so the loop continues to the next iteration.

3. Second Iteration:

- **Index:** `i = 1`
- **Element:** `arr[1] = 20`
- The condition `arr[i] == target` is checked. Here, `20 == 30` is `False`, so the loop continues to the next iteration.

4. Third Iteration:

- **Index:** `i = 2`
- **Element:** `arr[2] = 30`
- The condition `arr[i] == target` is checked. Here, `30 == 30` is `True`, so the function returns 2, which is the index of the target element.

5. Output:

- The returned index 2 is stored in the variable `result`.
- `print("Element found at index:", result)` prints `Element found at index: 2`.

If the target element is not found, the function would return `-1`, indicating that the element is not present in the list.

PREPARED BY: Sahil Wagh (M.Tech)

3 Binary Search

3.1 Binary Search Algorithm: Binary search is a fast search algorithm that works on sorted arrays. It repeatedly divides the search interval in half. If the value is less than the middle element, it narrows the interval to the lower half; otherwise, it narrows it to the upper half.

Example:

```
1 def binary_search(arr, target):
2     """Perform binary search on a sorted array."""
3     low = 0
4     high = len(arr) - 1
5
6     while low <= high:
7         mid = (low + high) // 2
8         if arr[mid] == target:
9             return mid
10        elif arr[mid] < target:
11            low = mid + 1
12        else:
13            high = mid - 1
14
15    return -1
16
17 # Sorted array
18 arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
19 target = 7
20 print(binary_search(arr, target))
```

Explanation: The `binary_search_iterative` function performs a binary search without recursion. It uses a while loop to search for a target value in a sorted array by adjusting the search range based on comparisons.

Dry Run:

1. `binary_search_iterative([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 7)`

- Initial values: `low = 0, high = 9`
- `mid = (0 + 9) // 2 = 4`
- `arr[mid] = arr[4] = 5`
- Since `5 < 7`, set `low = mid + 1 = 5`

2. `binary_search_iterative([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 7)`

- Current values: `low = 5, high = 9`
- `mid = (5 + 9) // 2 = 7`
- `arr[mid] = arr[7] = 8`
- Since `8 > 7`, set `high = mid - 1 = 6`

3. `binary_search_iterative([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 7)`

- Current values: `low = 5, high = 6`
- `mid = (5 + 6) // 2 = 5`
- `arr[mid] = arr[5] = 6`
- Since `6 < 7`, set `low = mid + 1 = 6`

4. `binary_search_iterative([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 7)`

- Current values: `low = 6, high = 6`

PREPARED BY: Sahil Wagh (M.Tech)

- $\text{mid} = (6 + 6) // 2 = 6$
- $\text{arr}[\text{mid}] = \text{arr}[6] = 7$
- Since $7 == 7$, return 6

4 Recursion

Recursion is a programming technique where a function calls itself directly or indirectly to solve a problem. This approach breaks down a complex problem into smaller, more manageable sub-problems of the same type, solving each sub-problem with the same function. The process continues until a base case is reached, which stops the recursive calls. Recursion is often used in problems involving repetitive tasks, such as traversing data structures (e.g., trees, graphs), performing calculations (e.g., factorial, Fibonacci sequence), and dividing problems into smaller instances (e.g., binary search, quicksort).

4.1 Calculating Factorial of a Number: Example:

```
1 def factorial(n):
2     """Calculate the factorial of a number using recursion."""
3     if n == 0:
4         return 1
5     else:
6         return n * factorial(n - 1)
7
8 print(factorial(5))
```

Dry Run:

1. `factorial(5)` is called.
2. Since $n \neq 0$, computes $5 * \text{factorial}(4)$.
3. `factorial(4)`:
 - Since $n \neq 0$, computes $4 * \text{factorial}(3)$.
 - `factorial(3)`:
 - Since $n \neq 0$, computes $3 * \text{factorial}(2)$.
 - `factorial(2)`:
 - * Since $n \neq 0$, computes $2 * \text{factorial}(1)$.
 - * `factorial(1)`:
 - Since $n \neq 0$, computes $1 * \text{factorial}(0)$.
 - `factorial(0)`:
 - Since $n == 0$, returns 1.
 - Returns $1 * 1 = 1$.
 - * Returns $2 * 1 = 2$.
 - Returns $3 * 2 = 6$.
 - Returns $4 * 6 = 24$.
 - 4. Returns $5 * 24 = 120$.

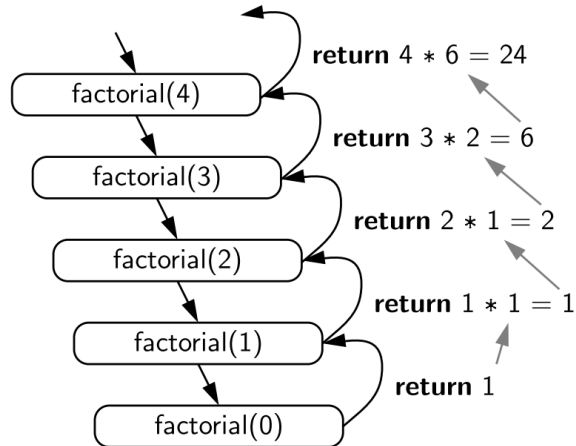


Figure 1: Flowchart of factorial computation.

4.2 Finding Maximum of an Array: Example:

```

1 def find_max(arr, index=0):
2     """Find the maximum element in an array using recursion."""
3     if index == len(arr) - 1:
4         return arr[index]
5     else:
6         max_of_rest = find_max(arr, index + 1)
7         return max(arr[index], max_of_rest)
8
9 arr = [3, 5, 7, 2, 8]
10 print(find_max(arr))

```

Dry Run:

1. `find_max([3, 5, 7, 2, 8], 0)` is called.
2. Since `index != 4`, it recursively calls `find_max([3, 5, 7, 2, 8], 1)`.
3. `find_max([3, 5, 7, 2, 8], 1)`:
 - Since `index != 4`, it recursively calls `find_max([3, 5, 7, 2, 8], 2)`.
 - `find_max([3, 5, 7, 2, 8], 2)`:
 - Since `index != 4`, it recursively calls `find_max([3, 5, 7, 2, 8], 3)`.
 - `find_max([3, 5, 7, 2, 8], 3)`:
 - * Since `index != 4`, it recursively calls `find_max([3, 5, 7, 2, 8], 4)`.
 - * `find_max([3, 5, 7, 2, 8], 4)`:
 - Since `index == 4`, it returns 8.
 - * Returns `max(2, 8) = 8`.
 - Returns `max(7, 8) = 8`.
 - Returns `max(5, 8) = 8`.
4. Returns `max(3, 8) = 8`.

4.3 Sum of an Array: Example:

```

1 def array_sum(arr, index=0):
2     """Calculate the sum of array elements using recursion."""
3     if index == len(arr):
4         return 0
5     else:
6         return arr[index] + array_sum(arr, index + 1)
7
8 arr = [1,3,5,7,9]
9 print(array_sum(arr))

```

Dry Run:

1. length=5
2. array_sum([1, 2, 3, 4, 5], 0) is called.
3. Since index != 5, it computes 1 + array_sum([1, 2, 3, 4, 5], 1).
4. array_sum([1, 2, 3, 4, 5], 1):
 - Since index != 5, it computes 2 + array_sum([1, 2, 3, 4, 5], 2).
 - array_sum([1, 2, 3, 4, 5], 2):
 - Since index != 5, it computes 3 + array_sum([1, 2, 3, 4, 5], 3).
 - array_sum([1, 2, 3, 4, 5], 3):
 - * Since index != 5, it computes 4 + array_sum([1, 2, 3, 4, 5], 4).
 - * array_sum([1, 2, 3, 4, 5], 4):
 - Since index == 5, it returns 0.
 - * Returns 4 + 0 = 4.
 - Returns 3 + 4 = 7.
 - Returns 2 + 7 = 9.
5. Returns 1 + 9 = 10.

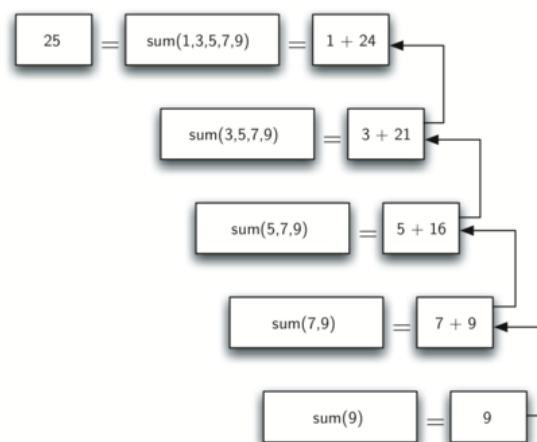


Figure 2: Flowchart of sum computation.

4.4 Reversing an Array: Example:

```
1 def reverse_array(arr, start, end=None):
2     """Reverse an array in place using recursion."""
3     if end is None:
4         end = len(arr) - 1
5     if start >= end:
6         return arr
7     arr[start], arr[end] = arr[end], arr[start]
8     return reverse_array(arr, start + 1, end - 1)
9
10 arr = [1, 2, 3, 4, 5]
11 print(reverse_array(arr, start=0))
```

Dry Run :

Let's go through each step of the recursion in detail:

1. Initial Call: reverse_array([1, 2, 3, 4, 5], 0)

- **Initial Values:** The function is called with `arr = [1, 2, 3, 4, 5]`, `start = 0`, and `end = None`.
- **Determine end:** Since `end` is `None`, it is set to `len(arr) - 1`, which is 4.
- **Comparison:** The condition `start < end` is checked. Here, `0 < 4` is `True`.
- **Swap Elements:** The elements at indices 0 and 4 are swapped, so `arr[0]` (which is 1) and `arr[4]` (which is 5) are exchanged. The array now becomes `[5, 2, 3, 4, 1]`.
- **Recursive Call:** The function is called recursively with `reverse_array([5, 2, 3, 4, 1], 1, 3)`.

2. Second Call: reverse_array([5, 2, 3, 4, 1], 1, 3)

- **Current Values:** The function is now working with `arr = [5, 2, 3, 4, 1]`, `start = 1`, and `end = 3`.
- **Comparison:** The condition `start < end` is checked again. Here, `1 < 3` is `True`.
- **Swap Elements:** The elements at indices 1 and 3 are swapped, so `arr[1]` (which is 2) and `arr[3]` (which is 4) are exchanged. The array now becomes `[5, 4, 3, 2, 1]`.
- **Recursive Call:** The function is called recursively with `reverse_array([5, 4, 3, 2, 1], 2, 2)`.

3. Third Call: reverse_array([5, 4, 3, 2, 1], 2, 2)

- **Current Values:** The function is now working with `arr = [5, 4, 3, 2, 1]`, `start = 2`, and `end = 2`.
- **Comparison:** The condition `start < end` is checked again. Here, `2 < 2` is `False`, but since the base case is `start >= end`, the function does not proceed to swap elements.
- **Return Statement:** Since `start >= end`, the function returns the current array `[5, 4, 3, 2, 1]`.

4. Returning from Recursion:

- The third call returns the array [5, 4, 3, 2, 1] to the second call.
- The second call then returns this array [5, 4, 3, 2, 1] to the first call.
- Finally, the first call returns the fully reversed array [5, 4, 3, 2, 1].

Final Output: The original array [1, 2, 3, 4, 5] is successfully reversed to [5, 4, 3, 2, 1].

4.5 Calculating Power of a Number: Example:

```
1 def power(base, exponent):
2     """Calculate base raised to the power of exponent using recursion."""
3     if exponent == 0:
4         return 1
5     else:
6         return base * power(base, exponent - 1)
7
8 print(power(2, 3))
```

Dry Run:

1. power(2, 3) is called.
2. Since exponent != 0, computes 2 * power(2, 2).
3. power(2, 2):
 - Since exponent != 0, computes 2 * power(2, 1).
 - power(2, 1):
 - Since exponent != 0, computes 2 * power(2, 0).
 - power(2, 0):
 - * Since exponent == 0, returns 1.
 - Returns 2 * 1 = 2.
 - Returns 2 * 2 = 4.
4. Returns 2 * 4 = 8.

4.6 Recursive Binary Search: Binary search can also be implemented using recursion. Here's how the recursive version works:

Example:

```
1 def recursive_binary_search(arr, target, low, high):
2     """Perform binary search on a sorted array using recursion."""
3     if low > high:
4         return -1
5     mid = (low + high) // 2
6     if arr[mid] == target:
7         return mid
8     elif arr[mid] < target:
9         return recursive_binary_search(arr, target, mid + 1, high)
10    else:
11        return recursive_binary_search(arr, target, low, mid - 1)
12
13 # Sorted array
14 arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
15 target = 7
16 print(recursive_binary_search(arr, target, 0, len(arr) - 1))
```

PREPARED BY: Sahil Wagh (M.Tech)

Explanation: The `binary_search` function recursively searches for a target value in a sorted array. It divides the array into two halves and continues searching in the relevant half based on the comparison.

Dry Run:

1. `binary_search([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 0, 9, 7)`

- `low = 0, high = 9`
- `mid = (0 + 9) // 2 = 4`
- `arr[mid] = arr[4] = 5`
- Since $5 < 7$, recurse with `binary_search([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5, 9, 7)`

2. `binary_search([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5, 9, 7)`

- `low = 5, high = 9`
- `mid = (5 + 9) // 2 = 7`
- `arr[mid] = arr[7] = 8`
- Since $8 > 7$, recurse with `binary_search([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5, 6, 7)`

3. `binary_search([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5, 6, 7)`

- `low = 5, high = 6`
- `mid = (5 + 6) // 2 = 5`
- `arr[mid] = arr[5] = 6`
- Since $6 < 7$, recurse with `binary_search([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 6, 6, 7)`

4. `binary_search([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 6, 6, 7)`

- `low = 6, high = 6`
- `mid = (6 + 6) // 2 = 6`
- `arr[mid] = arr[6] = 7`
- Since $7 == 7$, return 6